# Reto de recuperación — Corte 2

**Estudiante:** Miguel Pérez Ojeda

Computación en Internet 3 · NestJS + TypeORM + PostgreSQL + JWT

## Reglas

- Trabajas sobre **tu mismo repositorio del parcial**, en una rama nueva llamada `recuperacion`.
- Haz **un commit por cada punto** que completes, con un mensaje que diga qué hiciste. Se revisa el historial.
- La entrega se sustenta en persona (10 minutos): vas a explicar tu código y a hacer un cambio pequeño en vivo. Lo que no puedas explicar no cuenta completo.
- Usa imports relativos (`../student/entities/student.entity`), no `src/...`.

---

## Parte A — Corrige tu parcial (40 pts)

Tu proyecto no compilaba y el servicio quedó sin implementar. Esta parte es para que el módulo `program` funcione de punta a punta.

| # | Qué corregir | Pts |
|---|---|---|
| A1 | El proyecto compila: agrega los imports que faltan (`GetUser`, `User`, `PaginationDto`). | 4 |
| A2 | Registra `ProgramModule` en `AppModule`; en el módulo importa `TypeOrmModule.forFeature([Program])` y `UserModule`. | 6 |
| A3 | `code` con `@Matches(/^[A-Z]{3,6}$/)`. `@Min` y `@Max` son para números, no para textos. | 4 |
| A4 | Implementa `POST /api/program` (`201`, guarda `createdBy` con `@GetUser()`; repetido → `409`) y `GET /api/program` paginado. | 12 |
| A5 | Corrige la relación en `Student`: la función inversa del `@ManyToOne` es `(program) => program.students`, no `program.code`. | 6 |
| A6 | `POST /program` solo para `admin`: hoy `@Auth(ValidRoles.admin, ValidRoles.teacher)` deja pasar a un `teacher`, que debe recibir `403`. | 8 |

## Parte B — Módulo `faculty` (40 pts)

Los programas académicos pertenecen a una facultad. Una facultad tiene varios programas.

**Entidad `Faculty`:** `id` (UUID generado), `name` (texto, único), `code` (texto, único, 2 a 4 letras mayúsculas), `foundedYear` (entero), `createdBy` (`@ManyToOne(() => User)`).

| # | Requisito | Pts |
|---|---|---|
| B1 | Entidad y relación con `Program` (`@ManyToOne` en `Program`, `@OneToMany` en `Faculty`, columna `facultyId` en `program`). Módulo registrado. | 8 |
| B2 | `CreateFacultyDto`: `name` con `@MinLength(3)`, `code` con `@Matches(/^[A-Z]{2,4}$/)`, `foundedYear` con `@IsInt`, `@Min(1900)` y `@Max(2026)`. | 6 |
| B3 | `POST /api/faculty` (solo `admin`), `createdBy` con `@GetUser()`, `201`; nombre o código repetido → `409`. `GET /api/faculty` público y paginado. | 8 |
| B4 | `GET /api/faculty/:id` público: `ParseUUIDPipe` (`400`), trae sus programas y `404` si no existe. | 6 |
| B5 | `PATCH /api/faculty/:id/programs` (solo `admin`), body `{ "programIds": [...] }` (`@IsArray`, `@ArrayNotEmpty`, `@IsUUID('4', { each: true })`): acumulativo, en una transacción con `QueryRunner`; programa inexistente → `404` y ninguno asignado. | 12 |

## Sustentación (20 pts)

En persona, con tu código abierto. Se te pedirá explicar partes de tu solución y hacer un cambio pequeño en vivo.

## Entrega

URL del repositorio y nombre de la rama (`recuperacion`).